

Overall System Architecture

The system in question appears to be an application for providing chat responses based on company documentation using embeddings and AI services like Ollama. The architecture consists of several components that work together, each with specific responsibilities.

1. **Database Layer:** This is managed by ChromaDB, which stores documents, embeddings, and metadata.
2. **Processing Layer:** Handles the ingestion of text documents into ChromaDB and querying the database for relevant information.
3. **API Layer:** Provides a FastAPI endpoint to process incoming chat requests and return responses based on the provided context.

Detailed Analysis of Each File

app.py Purpose: Initializes a FastAPI app, connects to a ChromaDB collection, processes incoming queries through Ollama's embedding and chat services, and returns answers based on the provided context.

- **Inputs:** POST request with a JSON body containing a "question" string.
- **Outputs:** JSON response with an "answer" string.
- **API Endpoints:** /chat
- **External Services:** Ollama for embeddings and chat services
- **Important Functions:** chat, ollama.embeddings, ollama.chat

Dependencies: - Imports: fastapi, pydantic, chromadb, ollama - Depends on: QueryRequest class, ollama.embeddings, ollama.chat - Affects: qwen_test.py

ingest.py Purpose: Automatically processes and ingests text documents into a ChromaDB collection using Ollama embeddings.

- **Inputs:** Text files from the 'docs' directory.
- **Outputs:** Embeddings stored in a ChromaDB collection
- **External Services:** Ollama for generating text embeddings

Dependencies: - Imports: os, chromadb, ollama - No dependents, no imports (likely a standalone script)

qwen_test.py Purpose: Sends a chat request to the 'qwen2.5-coder:7b' model and prints the response content.

- **Inputs:** model as "qwen2.5-coder:7b" and messages as a list with a single user message "Hello".
- **Outputs:** The content of the model's response message
- **External Services:** Ollama.chat API

Dependencies: - Imports: ollama - Depends on: ollama.chat

test.py Purpose: Interact with a ChromaDB collection to retrieve documents.

- **Inputs:** None
- **Outputs:** List of documents from the 'company_docs' collection
- **Database Usage:** ChromaDB

Dependencies: - Imports: `chromadb` - No dependents, no imports (likely a standalone script)

test_chroma.py Purpose: Initializes and tests the ChromaDB library by creating a persistent client and collection.

- **Inputs:** None
- **Outputs:** Confirmation message indicating successful setup
- **Database Usage:** ChromaDB

Dependencies: - Imports: `chromadb` - No dependents, no imports (likely a standalone script)

test_rag.py Purpose: Demonstrates the process of embedding text and querying a collection using a vector database with an external embedding service.

- **Inputs:** A sample text to be embedded and a query embedding
- **Outputs:** Documents retrieved from the database that are most similar to the query embedding
- **External Services:** Ollama for text embedding

Dependencies: - Imports: `chromadb`, `ollama` - No dependents, no imports (likely a standalone script)

Dependency Analysis

The dependency analysis reveals the following:

1. **app.py** is at the center of the system with high in-degree due to its dependencies on other files.
2. **qwen_test.py** depends on **app.py**, indicating that it uses the FastAPI endpoint for sending chat requests.
3. **ingest.py** does not depend on any other file and can be considered a standalone script, likely for data ingestion purposes.

Bottlenecks

- **High In-Degree Files:** **app.py** has the highest in-degree, which suggests it is a critical component that handles many interactions.
- **Standalone Scripts:** **ingest.py**, **test.py**, **test_chroma.py**, and **test_rag.py** have no dependencies or dependents, indicating they might be standalone scripts for testing or data ingestion purposes.

Improvements

1. **Decentralize Testing:** Instead of using `qwen_test.py` to interact with the FastAPI endpoint, consider creating a separate testing module that simulates requests directly rather than relying on an external script.
2. **Separate Ingestion Logic:** If `ingest.py` is indeed a standalone script for data ingestion, ensure it has clear documentation and separation from other critical components to avoid potential bottlenecks.
3. **Refine External Service Handling:** Ensure that all calls to external services (like Ollama) are clearly documented and handled with proper error management to prevent system failures.

By addressing these issues, the overall system architecture can be made more robust and maintainable.